universität koblenz
weiter:denken

SoftLang Seminar -- Applications of AI (2026)

AI in Software Engineering

ML for Automated Software Testing

Manani, Bhavin Jitendrabhai

Matrikulation Number: **223201708**

WeST
Web Science and Technologies

RESEARCH QUESTION:

universität koblenz
weiter:denken

RQ: How have machine learning techniques been applied to automate software testing (e.g., test generation or prioritization), and what gains in fault detection or efficiency do they offer?

Why is it important?

- Software testing is time-consuming, costly and often performed manually.
- ML offers the potential to automate testing activities and improve software quality.
- Understanding its real impact supports better testing practices for industry and research.

What gap does it address?

- Existing literature is fragmented across different ML techniques and testing tasks.
- Findings on fault detection and efficiency are inconsistent and not well synthesized.
- A focused literature review provides a clearer, comparative view of the gains and open challenges.

Underlying assumption

- Machine learning techniques can automatically generate and prioritize test cases.
- These techniques lead to higher fault detection and more efficient testing compared to traditional approaches.

MOTIVATION

universität koblenz
weiter:denken

Why Automate Software Testing?

PROBLEMS

- Large software systems contain thousands of test cases.
- Running every test wastes time and computing resources.
- Manual prioritization is subjective and inefficient.
- ML enables intelligent, data-driven automation of testing.

MOTIVATION: The Testing Bottleneck

universität koblenz
weiter:denken

In modern CI/CD pipelines, **testing is one of the major bottlenecks** that slows down delivery.
AI/ML techniques can help prioritize what matters most and optimize testing effort.

THE PROBLEM

- CI/CD pipelines require rapid feedback.
- Running the full test suite takes hours or even days.
- Delayed feedback slows down releases and increases costs.

THE ML SOLUTION

- ML models predict which tests are most likely to fail.
- Only the most relevant tests are executed.
- Feedback time is drastically reduced.

IMPACT

- Faster feedback for developers
- Lower CI/CD costs and resource usage
- Higher productivity and better product quality

Traditional Testing vs. ML-assisted Testing (What the Literature Shows)

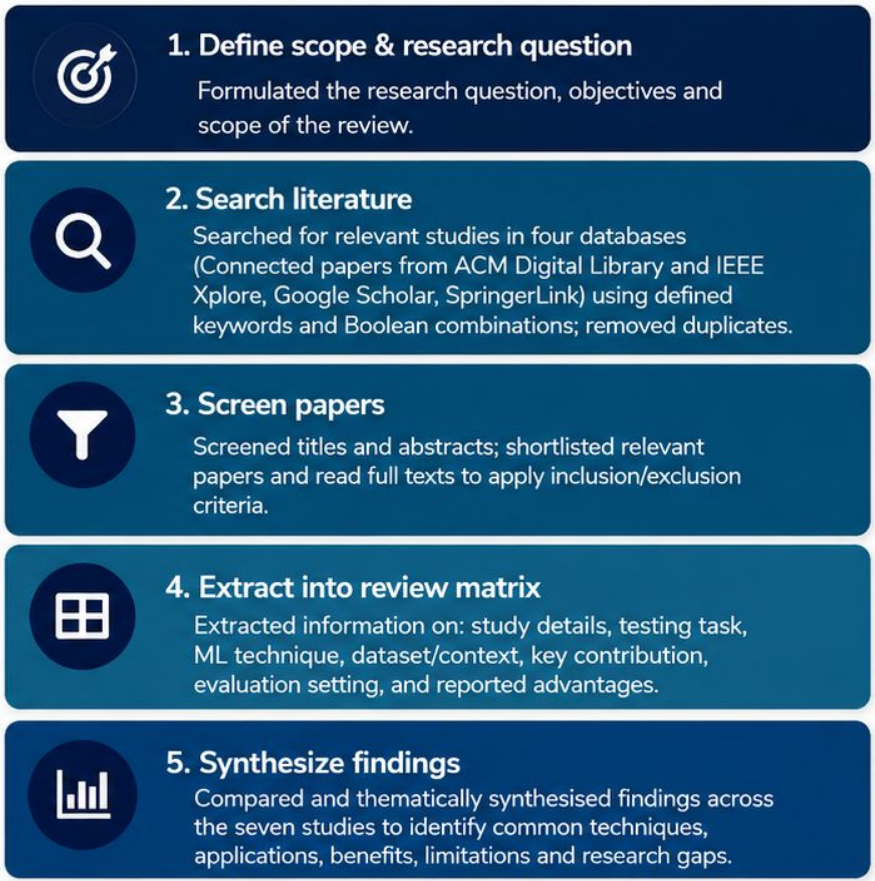
	Traditional Testing	ML-assisted Testing
Feedback Time	Slow feedback (all tests executed)	Faster feedback through test prioritization (earlier fault detection)
Execution Cost	High (unnecessary tests are executed)	Lower (only relevant tests are executed)
Test Selection	Manual / heuristic selection	Automated selection using ML models
Fault Detection	Defects detected later in the cycle	Defects detected earlier
Overall Outcome	Lower efficiency, higher effort	Better efficiency, higher productivity

Studies in our review show that ML-based test prioritization consistently achieves **earlier fault detection** and **reduces execution effort**, leading to faster and more efficient software delivery.

Takeaway | By focusing on the right tests, AI/ML techniques remove the testing bottleneck, enabling faster, cheaper, and more reliable software delivery.

METHODOLOGY: PAPER SELECTION

universität koblenz
weiter:denken



Search & selection at a glance

- **Databases searched:** ACM Digital Library, IEEE Xplore, Google Scholar, arXiv, Connected Papers
- **Keywords used:** "Machine learning", ”ML in software testing”, "reinforcement learning", "automated test case generation", “DL test prioritization”, "test prioritization", "fault detection", "software test efficiency", “test case selection using ML ”
- **Time frame:** 2019–2025
- **Language:** English
- **Study types:** Empirical studies and high-quality systematic reviews that apply ML to automate Software Testing.
- **Final selection: 7 core primary studies**
- **Selection approach:** Keyword search → Title → Abstract → Full text extraction → Text Screening using predefined **inclusion and exclusion criteria**

SEMI-SYSTEMATIC REVIEW

universität koblenz
weiter:denken

Type & source

Semi-systematic (narrative) literature review

Based on Snyder (2019) for conducting the review and van Wee & Banister (2024) for documenting the search and selection process.

Chosen because the objective was to identify, compare and synthesise how ML techniques are applied across software testing tasks and what benefits they provide.

Applications of ML in testing: Applied across test case prioritization (most common), test case generation, defect prediction/fault prediction, test selection, and flakiness prediction.

Inclusion criteria

- ML techniques applied to software testing (generation, prioritization, prediction, selection, etc.)
- Empirical evaluation with clear results
- Published in 2019–2024
- English language

Exclusion criteria

- Editorials, opinion pieces, tutorials
- Studies not related to software testing
- Duplicate publications
- Papers without empirical evaluation

- **Reported gains:** Consistent improvements in fault detection/defect prediction accuracy (up to ~15-25%), test execution time reduction (~10-40%), cost reduction, and overall testing efficiency.

- **Context factors:** Project size, domain (industrial vs. open-source), tooling support, and CI/CD environment affect effectiveness.

Outcome | Seven primary studies were synthesised to identify how machine learning techniques are applied across software testing tasks and the gains they provide in terms of efficiency, accuracy, fault prediction, automation and overall testing effectiveness.

ANALYSIS & DATA EXTRACTED

universität koblenz
weiter:denken

Paper	Information Extracted	Exact Extract & Location (Verification)	Contribution to Research Question
Zhao et al. 2023	• **Testing task:** Test Case Prioritization • **ML technique**: Machine Learning ranking model (MART) • **Key evidence**: Produces more effective prioritization than traditional approaches.	**Location:** Section 5, Para 2 **Extract:** "pretrained MART produces the optimal sequence on 80% subjects... crucial for achieving state-of-the-art prioritization efficiency."	Demonstrates how state-of-the-art ML models dramatically improve test execution sequencing and overall CI efficiency.
Sharif et al. 2021 (DeepOrder)	• **Testing task**: Test Case Prioritization • **ML technique**: Deep Neural Networks (DeepOrder) • **Key evidence**: Predicts failure probability to improve execution order.	**Location:** Section IV, Sub B, Para 3 **Extract:** "achieves superior early fault detection... increasing NAPFD over both random and FIFO sorting."	Demonstrates how Deep Learning leverages non-linear relationships to improve early fault detection over traditional methods.
Spieker et al. 2018 (Retecs)	• **Testing task**: Test Case Prioritization (TCP) • **ML technique**: Reinforcement Learning (Q-learning) • **Key evidence**: Priorities tests to detect failures earlier while reducing testing time.	**Location:** Section 4, Sub 4.3, Para 1 **Extract:** "RL agent learns to prioritize error-prone test cases... successfully minimizes the round-trip time between code commits and feedback."	Demonstrates how Reinforcement Learning automates test prioritization and minimizes developer wait-time.
Durelli et al. 2019 (Seed Paper)	• **Extracted**: Systematic Mapping Study; maps ML to Test Generation, Oracle evaluation, and Fault Prediction; highlights Supervised Learning (SVM, Decision Trees).	**Location:** Abstract & Section 3 **Extract:** "predict which parts of the software are most likely to contain bugs."	Explains why ML is effective and provides a taxonomy of where it performs best across the software testing lifecycle.
Afonso Fontes & Gay 2023	• **Study type**: Systematic Mapping Study • **Extracted**: Software testing activities, ML techniques, publication trends and research directions.	**Location:** Abstract / Results Summary **Extract:** "Supervised learning... and reinforcement learning are common... evaluated using metrics tied to the reward function."	Provides empirical evidence of which specific ML algorithms (Neural Networks vs RL) dominate specific automated testing tasks.
Bagherzadeh et al. 2022	• **Extracted**: Test Case Prioritization using Reinforcement Learning; comprehensive evaluation of state-of-the-art RL models in CI.	**Location:** Abstract & Conclusion **Extract:** "find an optimal ordering for the test case executions to detect faults as early as possible."	Provides broad empirical evidence that RL increases testing efficiency and fault detection performance across multiple datasets.
Pan et al. 2021 (Seed Paper)	• **Study type**: Systematic Literature Review • **Extracted**: Testing tasks, ML algorithms, datasets, evaluation metrics, research trends and limitations.	**Location:** Abstract / SLR Findings **Extract:** "combine information from partial and imperfect sources into accurate prediction models."	Complements empirical studies by providing an overview of existing ML features and identifying research gaps in CI.

KEY FINDINGS

universität koblenz
weiter:denken

#	Dimension	What is captured	Maps to RQ via	Example values from literature
1	**ML Technique / Method Type**	Type of ML approach used	Core comparison axis (ML vs. traditional)	Supervised Learning · Unsupervised Learning · Reinforcement Learning · Ensemble · Deep Learning · Hybrid Approaches
2	**Testing Task**	Specific automated testing activity addressed	Scope of automation in software testing	Test Case Prioritization · Test Case Generation · Defect Prediction · Failure Prediction · Test Selection · Flakiness Prediction
3	**Dataset / Data Source**	Type and source of data used	Context & generalizability of findings	Open Source Projects · GitHub Repositories · Bug Reports · Code Changes · CI/CD Logs · Industrial Datasets
4	**Evaluation Setting**	How the approach was evaluated	Ensures comparability across studies	Cross-Project Validation · Time-based Split · Within-Project Split · k-Fold CV · Hold-out Set · Simulation / Replay
5	**Performance / Benefits**	Gains achieved in fault detection or efficiency	Direct answer to "what gains" in the RQ	APFD ↑ · F-measure ↑ · Fault Detection ↑ · Execution Time ↓ · Cost ↓ · Effectiveness ↑
6	**Baseline / Comparator**	Method compared against	Strengthens evidence of gains	Random · Total Order · Greedy · Manual Priority · Traditional Heuristics · Other ML Models
7	**Key Factors / Context**	Conditions affecting performance of ML approaches	Explains why effects vary across studies	Dataset Size · Project Size · Domain · Imbalance · Feature Set · Tooling · CI Environment
8	**Limitations / Challenges**	Constraints and issues reported	Interprets mixed or limited effects	Data Quality · Overfitting · Generalizability · Reproducibility · Computational Cost · Lack of Standard Metrics

Overall Insight: ML-based testing techniques, especially test case prioritization and defect prediction, consistently improve efficiency and fault detection compared to traditional methods, though results depend on context, data, and evaluation design.

LIMITATION

universität koblenz
weiter:denken

Reproducibility Crisis

79% of TSP studies not reproducible (Pan 2022). Over 50% of all ML-testing papers do not share code or data. Most gains cannot be independently verified.

Pan 2022 — Section 7, p.29

Toy Dataset Problem

44% of ML test generation papers evaluate only on synthetic examples (Fontes 2023). Performance on real industrial systems remains largely unproven at scale.

Fontes 2023 — Section 4.7

Language Bias

ML models are mostly C/C++ and Java-specific. Very few language-agnostic approaches exist. Transfer to new codebases degrades performance significantly.

Shimmi 2025 — Section 7.3

Real-World Performance Gap

Models trained on controlled datasets may drop significantly in performance on real-world code. Same risk applies to ML-based testing tools as found for vulnerability detection.

Shimmi 2025 — Section 7.8

CONCLUSION & FUTURE WORK

universität koblenz
weiter:denken

Conclusion

ML is widely applied in software testing.
Mainly in test case prioritization, followed by generation, selection and defect prediction.

Clear benefits are achieved.
Early fault detection, reduced execution time and improved testing efficiency—especially in CI environments.

Reinforcement Learning & Deep Learning show strong potential.
Models trained on historical CI data and pre-trained models deliver promising results.

Limitations remain.
Small datasets, lack of standard metrics and limited real-world validation restrict generalization.

Overall takeaway.
ML effectively automates testing and improves quality, but stronger evidence and adoption are still needed.

Future Work

01 **Stronger datasets**
Build large, diverse and open datasets across projects and domains.

02 **Standardised evaluation**
Use common metrics and protocols for fair comparison of ML approaches.

03 **Real-world validation**
Conduct long-term industrial studies to measure impact in CI/CD pipelines.

04 **Hybrid & explainable models**
Combine ML with heuristics and use explainable AI to increase transparency and trust.

Bottom line

Machine Learning has clear potential to enhance automated software testing.
The next step is stronger data, standardisation and real-world adoption.

REFERENCES

universität koblenz
weiter:denken

#	Source	Venue / Year	DOI:
1	**Durelli et al.** — *The SPACE of Developer Productivity* (Seed Paper)	IEEE Transactions on Reliability **2019**	10.1109/TR.2019.2892517
2	**Pan et al.** — *Test Case Selection and Prioritization Using Machine Learning: A Systematic Literature Review* (Seed Paper)	Empirical Software Engineering **2021**	10.1007/s10664-021-10066-6
3	**Afonso Fontes & Gay** — *Artificial Intelligence for Software Testing: A Systematic Mapping Study*	arXiv **2023**	10.48550/arXiv.2206.10210
4	**Spieker et al.** — *Reinforcement Learning for Automatic Test Case Prioritization and Selection in Continuous Integration (Retecs)*	ICSE **2018**	10.1145/3092703.3092709
5	**Sharif et al.** — *DeepOrder: Deep Learning for Test Case Prioritization*	ICSME **2021**	10.1109/ICSME52107.2021.00053
6	**Bagherzadeh et al.** — *Reinforcement Learning for Test Case Prioritization: A Comprehensive Study*	arXiv **2022**	10.48550/arXiv.2011.01834
7	**Zhao et al.** — *Revisiting Machine Learning-based Test Case Prioritization for Continuous Integration*	arXiv **2023**	10.48550/arXiv.2311.13413

GenAI DECLARATION

universität koblenz
weiter:denken

In the peparation of this seminar and presentation, Generative AI tools were used as supportive aids. All ideas, analysis, and conclusions are the original work of the author.

AI Platform / Tool	Purpose of Use	How It Was Used	My Contribution
ChatGPT (OpenAI)	Brainstorming, structuring content, and general guidance	• Ideation and outline creation • Concept explanations • Clarifying research questions	Final content selection, critical evaluation, and original writing
Gemini (Google)	Content summarization and information synthesis	• Summarizing research papers • Comparative points extraction • Data interpretation support	Verification with original sources and adding own analysis
Claude (Anthropic)	Deep explanations and refining complex topics	• Understanding technical concepts • Alternative perspectives • Refining research insights	Critical review and integration of insights into the presentation
Perplexity AI	Literature search and fact finding	• Finding relevant papers and sources • Quick fact verification • Exploring related work	Source validation and selection of relevant references
Grammarly	Formal writing, grammar and sentence structure	• Grammar and spelling check • Sentence restructuring • Improving clarity and tone	Final review and ensuring academic tone and consistency
Canva AI	Presentation design and visual generation	• Slide layout suggestions • Image and icon recommendations • Visual enhancements	Final slide design, customization, and content ownership

Declaration

I confirm that Generative AI tools were used ethically and responsibly as supportive aids only.
All information has been verified from original sources, and the final content is my own work.

Thank you for your Attention!

Time for Questions and Discussions.